

程式內部架構(現況)—— index.html 裡面裝了什麼

整包工具是一個檔:11,270 行 = 851 行 CSS + 986 行畫面標記 + 9,601 行 JavaScript。下面是那 9,601 行的分法,所有行數都是實際數出來的。

index.html

沒有 build、沒有框架、沒有 npm 相依 —— 打開就是它

① 畫面層(UI)

986 行標記 + 851 行 CSS + 1,738 行 JS

10 個分頁

匯入 · 附加費 · 尺寸 · 通路 · 需求

歷史 · 代碼 · 儀表板 · 設定 · Admin

設定 UI 1,747 行

與桌面版共用的設定模板

附加費 · 尺寸 · 通路 · Demand 表單

外觀與體驗 1,738 行

主題 · 雙語 · 倉庫場景 · 天色天氣

跟計價完全無關

② 計價引擎 2,234 行 —— 這份工具真正的資產

提到 ME / AUTH / supabase / localStorage 的次數:0

讀帳單 68

認出格式、切成一列一件

xlsx / xls / 無標題 csv

欄位位置 157

哪一欄是重量、哪一欄

是區、哪一欄是說明

計費重量 178

材積重 vs 實重,取大

AHS / 大件的最低重量

附加費 912 最大一塊

AHS · 大件 · 住宅 · 申報

價值 · 偏遠 · 簽收...

分層 220

同一件的好幾列怎麼併

FRT / 更正 / 退運

燃油 349

燃油率,以及附加費

要不要一起算燃油

Demand 旺季 113

四個子表,依週次

和件量級距查

自訂附加費 178

你自己加的規則

不在 UPS 的表裡

③ 報表輸出與儀表板 2,504 行

吃引擎吐出來的逐筆結果,不自己算價

重計價報表 104

reportRows() —— 匯出與

儀表板共用的同一份數字

匯出 xlsx 852

客戶報表 + 內部對帳單

由 xlsx.js 寫檔

儀表板與圖表 1,409

期別選單、趨勢、佔比

圖是自己畫的 SVG,沒用套件

④ 資料層 531 行 —— 東西存在這台電腦裡

localStorage 44 處 · IndexedDB 3 處 · sessionStorage 4 處

跨期歷史 261

每一期歸檔、跨期更正、

同一件在不同期的比對

儲存空間統計 270

告訴你瀏覽器裡存了多少

哪一期佔了多少空間

實際存放

localStorage 設定 · 索引

IndexedDB 逐筆明細

⑤ 身分層 738 行 —— 五層裡唯一要換的一層

全檔 34 次 ME · 15 次 LOCAL_MODE 分支

登入

LOCAL_MODE = 沒有 authMode

LOCAL_ACCOUNTS[]

帳號 + sha256 雜湊 + 角色

login()

算 sha256(帳號:密碼) 去比對

authIdentity() / setMe()

組出 ME = { id, role, name, sync }

applyRole()

決定 Admin 分頁露不露出來

Supabase 那條路的程式碼也在這裡,

只是沒有被走到

★ 接縫 —— 身分只從這裡外流

whoKey() → ME.id 或 "anon"

histKey() → 期別索引

histRuleKey() → 歷史規則

histInvKey() → 帳單清單

idbName() → IndexedDB 名

cfgKeys() → 費率設定

六個都是同一個公式:

原本的 key + "::-" + whoKey()

身分一換 → 前綴全換 →

畫面上東西「不見了」

(資料還在,在舊前綴底下)

雲端同步與帳號管理

cfgPush() / cfgPull()

費率設定上下雲,鑰匙 = ME.sync

histCloudPush / Pull / Delete

帳單歷史,先 gzip 再上傳

claimLegacy()

認領「分帳號之前」的共用歷史

—— 注意:不是身分搬遷

adminCall() / recordLogin()

loadUsers() / loadLoginHistory()

這四個要有 Supabase 登入才會動,

現在等於閒置

1 個檔案 · 11,270 行

重點一:身分是怎麼滲進整個程式的

全檔提到 ME. 有 34 次,聽起來像散得到處都是。實際上其中 24 次就在身分層自己裡面,引擎和資料層合計只有 5 次,UI 只有 2 次。

真正把身分帶進資料的,只有左邊那 6 個函式。

這件事有兩個意思,一好一壞:

壞的:換身分 = 換前綴。費率、期別索引、帳單清單、IndexedDB 全部一起改名,畫面上看起來像資料被清空了。

好的:要補的搬遷程式很小、很明確 —— 就是把這 6 個前綴的舊值搬到新值,一個地方改完全部生效。不是散在 34 個地方要慢慢找。

另外有一個小瑕疵:cfgKeys() 沒有呼叫 whoKey(),而是自己把同一行邏輯再寫一次。現在結果一樣,但那是兩個真相來源。

要動身分之前,先把它改成呼叫 whoKey(),讓接縫真的只有一條。

重點二:計價引擎不知道有沒有登入這回事

第 1,670 到 3,145 行,共 1,476 行。在這段裡搜尋 ME、AUTH、SYNC、supabase、LOCAL_MODE、localStorage、indexedDB:

命中 0 次。

它只吃兩樣東西 —— 一份帳單、一份費率表;吐一份逐筆結果。誰在用、有沒有登入、資料存在哪,它完全不知道,也不需要知道。

所以換登入方式,動不到它一個字。

這是刻意的,也是這份工具最該保住的性質。

檔案外面的東西

瀏覽器本身提供的
localStorage · IndexedDB · sessionStorage
WebCrypto(算 sha256,只在 https 有)

另外載入的兩個檔
supabase-js(連雲端) · xlsx(讀寫 Excel)

設定
auth-config.js —— 三個值,決定走哪條登入

雲端
Supabase RPC ×6 · Edge Function ×2

程式內部架構(理想)—— 只動第五層,其他四層一個字都不改

灰色 = 完全不動。綠色 = 已經寫好,只差設定。橘色 = 要補的三塊。這不是重寫,是把身分那一層抽乾淨,然後補上三個缺口。

index.html

仍然是一個檔。理想不是拆檔案,是把一層的責任講清楚

① 畫面層(UI) 3,575 行0 行改動

10 個分頁・設定表單・主題・雙語・倉庫場景 —— 它只問「現在是誰、是不是主帳號」,不管那個答案是哪裡來的

② 計價引擎 2,234 行0 行改動

讀帳單・欄位位置・計費重量・附加費・分層・燃油・Demand・自訂附加費 —— 本來就不知道有登入這回事

③ 報表輸出與儀表板 2,504 行0 行改動

重計價報表・匯出 xlsx・儀表板與圖表 —— 吃引擎吐出來的結果,跟身分無關

④ 資料層 531 行一處要改

跨期歷史・儲存空間統計・localStorage / IndexedDB 的實際讀寫 —— 存法不變,前綴照樣從接縫拿

▲ 缺口 2:帳單歷史的雲端同步目前寫死只在本機模式跑。要把它接到身分層,讓兩種模式都同步。

⑤ 身分層 —— 唯一要動的一層。做法:把「身分從哪來」和「身分怎麼用」切開

5a 身分提供者:一個小介面

上面四層要的其實只有這幾樣東西:

{ id, role, name, sync?,
signIn(), signOut(), restore() }

把它講清楚,底下換什麼實作都行,
上面完全不用知道。

現在是靠散在各處的 LOCAL_MODE 判斷達成同一件事。

5b ★ 接縫 —— 完全不變,這正是重點

whoKey()・histKey()・histRuleKey()
histInvKey()・idbName()・cfgKeys()

六個函式一行都不用改。整個程式碰身分只有
這一條路,所以換掉底下的實作是安全的。

▲ 唯一要順手修的:cfgKeys() 目前自己重算
一次 ME.id,改成呼叫 whoKey(),來源只留一個。

5c 實作一:本機帳號(留著當退路)

LOCAL_ACCOUNTS[] + sha256(帳號:密碼)
id = "local::<帳號>"

不刪掉。切換出問題時,把 authMode 拿掉就退
回這條路,不用回滾程式碼。

現在就是走這一條

5d 實作二:Supabase Auth(程式已經寫好)

signInWithPassword() → JWT
id = 帳號的 UUID,role 從 app_metadata 來
費率改存 user_settings,登入寫 login_history,
帳號管理走 admin-users

在 auth-config.js 加一行 authMode 就走這一條

▲ 缺口 1:身分搬遷 —— 這一層唯一真的要新寫的東西

換實作就換 id,換 id 就換前綴。第一次用新身分登入時要做一次:如果新前綴底下是空的,而這台電腦上有同名帳號的舊資料,
就把六個前綴的內容搬過來 —— 費率設定、期別索引、歷史規則、帳單清單,以及整個 IndexedDB 資料庫。

好消息:形狀已經有現成的了。claimLegacy() 做的就是同一件事(認領「分帳號之前」的共用歷史),照著擴充即可。

要注意 claimLegacy() 裡那個判斷:只有主帳號會收。共用電腦上,子帳號登入一次就把整台機器的歷史吸走 —— 搬遷也要有同樣的防線。

要改的:5 層裡的 1 層

要補的三塊,以及各自的份量

1・身分搬遷

照 claimLegacy() 的形狀寫一個,把六個前綴
的舊值搬到新值。集中在一個函式裡。
不寫也能上線,但每個人第一次登入都要自己
匯出再匯入設定。

2・帳單歷史的雲端同步

histCloudPush / Pull / Delete 現在開頭就擋掉
非本機模式。需要一張依 auth.uid() 的表加
RLS,做 local_history 已經在做的事。
不補的話,切過去等於這個功能退掉。

3・兩個小整理

cfgKeys() 改成呼叫 whoKey(),接縫只留一條;
刪掉 authconfig.js —— 沒有任何地方載入它,
留著只會讓人改錯檔。

三塊都很小。難的不是寫,是記得要寫。

為什麼可以只動一層

因為這份程式碼其實已經分層了,只是沒有把
界線寫下來:

- 引擎不碰身分,一次都沒有
- UI 只問「是誰、是不是主帳號」
- 資料層只透過 6 個函式拿前綴

所以「理想」不是把它拆成十個檔案、加一套
框架。真正的理想是:把已經存在的那條界線
講明白,讓底下換實作變成一件無聊的事。

一個檔 11,270 行不是問題。界線模糊才是。

改完怎麼驗收

① 第 1,670-3,145 行的 diff 是空的。引擎沒被
碰到,計價結果不可能變。

② 用舊帳號在同一台電腦切過去,登入後費率
和帳單歷史自己出現,不用手動匯入。

③ 在 A 電腦歸檔一期,B 電腦登入後看得到。

④ 把 authMode 那一行拿掉,重新部署,一切
回到今天的樣子,舊資料也還在。

⑤ 共用電腦上,子帳號登入不會把別人的歷史
吸到自己名下。
五條都過,才算真的換完。